UT5. Diseño Físico de Bases de Datos

Lenguaje de Manipulación de Datos (DML) (INSERT - UPDATE - DELETE)

INTRODUCCIÓN

El DML (*Lenguaje de Modificación de Datos*) es una de las partes fundamentales del lenguaje SQL. Lo forman las instrucciones capaces de modificar (añadir, cambiar o eliminar) los datos de las tablas. Al conjunto de instrucciones DML que se ejecutan consecutivamente, se le llama **transacción**.

Lo interesante de las transacciones es que podemos anularlas, ya que forman una unidad lógica de trabajo que hasta que no se acepten, sus resultados no serán definitivos. En todas las instrucciones DML, el único dato devuelto por el sistema es el número de filas que se han modificado al ejecutar la instrucción.

Las **sentencias DML** del lenguaje SQL son las siguientes:

- La sentencia **SELECT**, que se utiliza para extraer información de la base de datos, ya sea de una/s tabla/s.
- La sentencia **INSERT**, cuyo cometido es insertar uno o varios registros en alguna tabla.
- La sentencia **DELETE**, que borra registros de una tabla.
- La sentencia **UPDATE**, que modifica registros de una tabla.

En este tema nos vamos a centrar en el uso de las sentencias **INSERT, UPDATE y DELETE.**

INSERCIÓN DE DATOS

Añadir datos a una tabla se realiza mediante la instrucción **INSERT.** Su sintaxis fundamental es:

```
INSERT INTO {nombre_tabla | nombre_vista } [(columna1 [, columna2]...)]
VALUES (valor1 [, valor2] ... );
```

Notación: la lista de columnas en las que insertamos es opcional, por lo que va entre corchetes. Si no se especifica se esperan valores para todas las columnas.

Donde

- **NombreTabla**..... es el nombre de la tabla en la que queremos insertar una fila.
- **NombreColumna**..... Incluye las columnas en las que queremos insertar.
- **Valores**..... indica los valores que se insertarán, existiendo una correspondencia con la lista de columnas anterior

INSERCIÓN DE DATOS

Analizando el formato de INSERT se pueden insertar filas en una tabla sin especificar la lista de Columnas pero en este caso la lista de valores a insertar deberá coincidir en número y posición con las columnas de la tabla. El orden establecido par las columnas será el indicado al crear las tablas.

Recordar: para conocer ese orden basta invocar al comando **DESCRIBE**

Las columnas no rellenadas explícitamente con la orden INSERT, se rellenan con su valor por defecto (**DEFAULT**) o bien con **NULL** si no se indicó valor por defecto alguno.
Si alguna columna tiene restricción de obligatoriedad (**NOT NULL**), ocurrirá un error si no indicamos un valor para dicha columna.

INSERCIÓN DE DATOS

Por ejemplo, supongamos que tenemos una tabla de clientes
Campos: *dni*, *nombre*, *apellido1*, *apellido2*, *localidad* y *dirección*

```
INSERT INTO clientes VALUES( '11111111','Pedro','Gutiérrez', 'Crespo',DEFAULT,NULL);
INSERT INTO clientes(dni,nombre,apellido1,apellido2)
VALUES('11111111','Pedro','Gutiérrez', 'Crespo');
```

Son equivalentes puesto que, en la segunda instrucción, los campos no indicados se rellenan con su valor por defecto y la dirección no tiene valor por defecto.

Los valores por defecto se indican, durante la creación o modificación de la estructura de una tabla, a través de la palabra clave **DEFAULT**.

Lenguaje de Manipulación de Datos

Supongamos que tenemos el siguiente diseño físico de una tabla:

```
CREATE TABLE EMPLEADOS (
  COD       NUMBER(2)     PRIMARY KEY,
  NOMBRE    VARCHAR2(50) NOT NULL,
  LOCALIDAD VARCHAR2(50) DEFAULT 'Écija',
  FECHANAC  DATE
);
```

La forma más habitual de introducir datos es la siguiente:

```
INSERT INTO EMPLEADOS VALUES (1, 'Pepe', 'Osuna', '01/01/1970');
INSERT INTO EMPLEADOS VALUES (2, 'Juan', DEFAULT, NULL);
INSERT INTO EMPLEADOS VALUES (3, 'Sara', NULL, NULL);
```

Es obligatorio introducir valores para los campos COD y NOMBRE. Dichos campos no pueden tener valor NULL. Podemos insertar sólo el valor de ciertos campos. En este caso hay que indicar los campos a insertar y el orden en el que los introducimos:

```
INSERT INTO EMPLEADOS(NOMBRE, COD) VALUES ('Ana', 5);
```

INSERCIÓN DE DATOS

Ejemplo de **inserción con un campo autonumérico**:

```
#Borramos por si existe la BD
DROP DATABASE IF EXISTS tienda_inventario;
CREATE DATABASE tienda_inventario;
USE tienda_inventario;
#Creamos la tabla inventario
CREATE TABLE inventario (
    num INT(2) AUTO_INCREMENT PRIMARY KEY,
    descripcion VARCHAR(15)
    );
```

1.- Inserción sin enumerar la columna autonumérica.

INSERT INTO inventario (descripcion)
VALUES ("MESA");

2.- Inserción de toda la fila (en este caso debe ponerse NULL en la columna correspondiente)

INSERT INTO inventario
VALUES (NULL,"SILLA MADERA");

INSERCIÓN DE DATOS

Inserción de datos obtenidos de una consulta

También es posible insertar datos en una tabla que hayan sido obtenidos de una consulta realizada a otra tabla/vista u otras tablas/vistas. Su forma es:

```
INSERT INTO table2 (column1, column2, column3, ...)
SELECT column1, column2, column3, ...
FROM table1
WHERE condition;
```

INSERCIÓN DE DATOS (REPLACE)

Un problema frecuente a la hora de insertar filas es que intentemos añadir un valor que ya existe, en una columna que no admita duplicados.

Existe una sentencia para añadir filas que supone una alternativa para la sentencia **INSERT**. Se trata de la sentencia **REPLACE** que funciona igual que **INSERT** excepto en el caso de columnas **PRIMARY KEY** o **UNIQUE**.

La **sintaxis de esta sentencia** es:

REPLACE [INTO] NombreTabla VALUES (Valor1, Valor2, ...) , (....)

También se puede utilizar:

REPLACE [INTO] NombreTabla SET Columna1 = Valor1, Columna2 = Valor2, ...

Con SELECT:

REPLACE [INTO] NombreTabla1, [(Columna1, [, Columna2] ...)] SELECT {Columna1 [, Columna2] ... | *} FROM NombreTabla2 [CLÁUSULAS DE SELECT];

Ejemplo: `REPLACE INTO inventario VALUES (1,"MESA MADERA");`

MODIFICACIÓN DE DATOS

En ocasiones necesitaremos modificar alguno de los datos de las filas existentes de una tabla. Por ejemplo, cambiar el salario o el departamento de uno o varios empleados, etc. En estos casos utilizaremos la sentencia UPDATE.

La sentencia de modificación de filas es UPDATE cuyo formato genérico es el siguiente:

```
UPDATE {nombre_tabla | nombre_vista}
SET columna1=valor1 [, columna2=valor2] ...
[WHERE condición];
```

Notación: puede actualizarse una o varias columnas, por lo que la segunda actualización va entre corchetes La cláusula WHERE aparece entre corchetes porque es opcional. En el caso de que no se utilice, la actualización afectará a toda la tabla.

Donde

- **NombreTabla**..... indica la tabla destino donde se encuentran las filas que queremos borrar.
- **NombreColumna**..... indica el nombre de la columna cuyo valor queremos modificar.
- **Valores**..... es una expresión cuyo valor resultado será el nuevo valor de la columna.
- **Condición**..... es la condición que deben cumplir las filas para que les afecte la modificación.

MODIFICACIÓN DE DATOS

Partimos de la tabla socios

```
mysql> SELECT  * FROM socios;
+----------+---------------+-----------+------------+----------------+---------------+
| socio_no | apellidos     | telefono  | fecha_alta | direccion      | codigo_postal |
+----------+---------------+-----------+------------+----------------+---------------+
|     1000 | LOPEZ SANTOS  | 916543267 | 2005-01-08 | C. REAL 5      |         28400 |
|     1001 | PEREZ CERRO   | 918451256 | 2005-01-12 | C. MAYOR 31    |         28400 |
|     1002 | LOPEZ PEREZ   | 916543267 | 2005-01-18 | C. REAL 5      |         28400 |
|     1003 | ROMA LEIVA    |           | 2005-01-21 | C. PANADEROS 9 |         28431 |
|     1004 | GOMEZ DURUELO | 918654329 | 2005-01-31 | C. REAL 15     |         28400 |
|     1005 | PEÑA MAYOR    | 918515256 | 2000-01-01 | C. LARGA 31    |         28431 |
+----------+---------------+-----------+------------+----------------+---------------+
```

1 - Supongamos que se desea cambiar la dirección del socio de numero 1000 y la nueva dirección es :

```
mysql> UPDATE socios
    -> SET direccion = 'C.CUESTA 2'
    -> WHERE socio_no = 1000;
Query OK, 1 row affected (0.08 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

MODIFICACIÓN DE DATO

```
mysql> SELECT  * FROM socios;
+----------+---------------+-----------+------------+----------------+---------------+
| socio_no | apellidos     | telefono  | fecha_alta | direccion      | codigo_postal |
+----------+---------------+-----------+------------+----------------+---------------+
|     1000 | LOPEZ SANTOS  | 916543267 | 2005-01-08 | C. REAL 5      |         28400 |
|     1001 | PEREZ CERRO   | 918451256 | 2005-01-12 | C. MAYOR 31    |         28400 |
|     1002 | LOPEZ PEREZ   | 916543267 | 2005-01-18 | C. REAL 5      |         28400 |
|     1003 | ROMA LEIVA    |           | 2005-01-21 | C. PANADEROS 9 |         28431 |
|     1004 | GOMEZ DURUELO | 918654329 | 2005-01-31 | C. REAL 15     |         28400 |
|     1005 | PEÑA MAYOR    | 918515256 | 2000-01-01 | C. LARGA 31    |         28431 |
+----------+---------------+-----------+------------+----------------+---------------+
```

2. Supongamos que se desea cambiar el teléfono del socio de número 1000 y el nuevo teléfono es 918455431 y la dirección anterior.

```
mysql> UPDATE socios
    -> SET telefono = '918455431',direccion='C.CUESTA 2'
    -> WHERE socio_no = 1000;
Query OK, 0 rows affected (0.08 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

MODIFICACIÓN DE DATO

```
mysql> SELECT  * FROM socios;
+----------+---------------+-----------+------------+----------------+---------------+
| socio_no | apellidos     | telefono  | fecha_alta | direccion      | codigo_postal |
+----------+---------------+-----------+------------+----------------+---------------+
|     1000 | LOPEZ SANTOS  | 916543267 | 2005-01-08 | C. REAL 5      |         28400 |
|     1001 | PEREZ CERRO   | 918451256 | 2005-01-12 | C. MAYOR 31    |         28400 |
|     1002 | LOPEZ PEREZ   | 916543267 | 2005-01-18 | C. REAL 5      |         28400 |
|     1003 | ROMA LEIVA    |           | 2005-01-21 | C. PANADEROS 9 |         28431 |
|     1004 | GOMEZ DURUELO | 918654329 | 2005-01-31 | C. REAL 15     |         28400 |
|     1005 | PEÑA MAYOR    | 918515256 | 2000-01-01 | C. LARGA 31    |         28431 |
+----------+---------------+-----------+------------+----------------+---------------+
```

3. Si no se incluye la clausula WHERE la actualización afectará a todas las filas. El siguiente ejemplo modifica la fecha de alta de todos los empleados al valor 1 de enero de 2005.

```
mysql> UPDATE socios
    -> SET fecha_alta = '2005-01-01';
Query OK, 6 rows affected (0.06 sec)
Rows matched: 6  Changed: 6  Warnings: 0
```

MODIFICACIÓN DE DATOS

En la condición se pueden utilizar cualquiera de los operandos de comparación.

Operador	Significado
>	Mayor que
<	Menor que
>=	Mayor o igual que
<=	Menor o igual que
=	Igual
<>	Distinto
!=	Distinto

Además se puede utilizar:

Operador	Significado
AND	Devuelve verdadero si las expresiones a su izquierda y derecha son ambas verdaderas
OR	Devuelve verdadero si cualquiera de las dos expresiones a izquierda y derecha del OR, son verdaderas
NOT	Invierte la lógica de la expresión que está a su derecha. Si era verdadera, mediante NOT pasa a ser falso.

-- Para los empleados que nacieron a partir de 1970 ponemos nombres en mayúscula y localidades Marchena

```
UPDATE EMPLEADOS
SET NOMBRE=UPPER(NOMBRE),
LOCALIDAD='Marchena'
WHERE FECHANAC >= '1970/01/01';
```

MODIFICACIÓN DE DATOS

También es posible realizar una actualización de datos a partir de una subconsulta.

-- Incrementar un 10% sueldo de los empleados que trabajen en la sección de Ventas.

UPDATE EMPLEADOS

SET sueldo = sueldo * 1.10

WHERE id_sección = (**SELECT** id_sección **FROM** secciones **WHERE** nom_sección LIKE ‘Ventas’

ELIMINACIÓN DE DATOS

Es más sencilla que el resto, elimina los registros de la tabla que cumplan la condición indicada.

Se realiza mediante la instrucción **DELETE**.

```
DELETE [ FROM ] {nombre_tabla|nombre_vista}
[WHERE condición] ;
```

Donde:

NombreTabla ….. Indica la tabla destino donde se encuentra las filas que queremos borrar.

Condición ….. Es la condición que deben cumplir las filas a borrar

Hay que tener cuidado con las claves ajenas. Si intentamos borrar un registro que está siendo utilizado como clave foránea nos dará un error. (SOLUCIÓN: FK ON DELETE CASCADE)

ELIMINACIÓN DE DATOS

```
-- Borramos empleados de Estepa
DELETE EMPLEADOS
WHERE LOCALIDAD='Estepa';

-- Borramos empleados cuya fecha de nacimiento sea anterior a 1970
-- y localidad sea Osuna
DELETE EMPLEADOS
WHERE FECHANAC < '01/01/1970' AND LOCALIDAD = 'Osuna';

-- Borramos TODOS los empleados;
DELETE EMPLEADOS;
```

CONTROL DE TRANSACCIONES: COMMIT Y ROLLBACK

Una transacción es un conjunto de tareas relacionadas que se realizan de forma satisfactoria o incorrecta como una unidad. En términos de procesamiento, **las transacciones se confirman o se anulan**.

Para que una transacción se confirme se debe garantizar la permanencia de los cambios efectuados en los datos. Los cambios deben conservarse aunque el sistema se bloquee o tengan lugar otros eventos imprevistos.

Existen 4 propiedades necesarias, que son conocidas como **propiedades ACID**:

- ➔ **Atomicidad**: Una transacción es una unidad de trabajo el cual se realiza en su totalidad o no se realiza en ningún caso.
- ➔ **Coherencia**: Una transacción es una unidad de integridad porque mantiene la coherencia de los datos, transformando un estado coherente de datos en otro estado de datos igualmente coherente.
- ➔ **Aislamiento**: Una transacción es una unidad de aislamiento, permitiendo que transacciones concurrentes se comporten como si cada una fuera la única transacción que se ejecuta en el sistema.
- ➔ **Permanencia**: Si una transacción se realiza satisfactoriamente, el sistema garantiza que sus actualizaciones se mantienen aunque el equipo falle inmediatamente después de la confirmación.

CONTROL DE TRANSACCIONES: COMMIT Y ROLLBACK

En términos más prácticos, una transacción está formada por una **serie de instrucciones DML**. Una transacción comienza con la primera instrucción DML que se ejecute y finaliza con una operación **COMMIT (**si la transacción se confirma) o una operación **ROLLBACK (**si la operación se cancela).

Hay que tener en cuenta que cualquier instrucción DDL o DCL da lugar a un COMMIT implícito, es decir todas las instrucciones DML ejecutadas hasta ese instante pasan a ser definitivas.

Cuando hacemos modificaciones en las tablas estas no se hacen efectivas (llevan a disco) hasta que no ejecutamos la sentencia COMMIT. Cuando ejecutamos comandos DDL (definición de datos) se ejecuta un COMMIT automático, o cuando cerramos la sesión.

El comando ROLLBACK no permite deshacer estos cambios sin que lleguen a validarse. Cuando ejecutamos este comando se deshacen todos los cambios hasta el último COMMIT ejecutado.

CONTROL DE TRANSACCIONES: COMMIT Y ROLLBACK

Hay dos formas de trabajar con **AUTO_COMMIT** activado (validación automática de los cambios) o desactivado. Si está activado se hace COMMIT automáticamente cada sentencia y no es posible hacer ROLLBACK. Si no lo está, tenemos la posibilidad de hacer ROLLBACK después de las sentencias DML (INSERT, UPDATE, DELETE) dejando sin validar los cambios. Es útil para hacer pruebas.

Existe una variable, **AUTO_COMMIT,** que indica la forma de trabajo y tiene el **valor 1** si está en modo AUTO_COMMIT y **0 si no lo está**. Por defecto el MySQL está en modo AUTO_COMMIT. Su valor se puede cambia con la sentencia:

mysql> SET AUTOCOMMIT = 0 ó SET AUTOCOMMIT = 1;

CONTROL DE TRANSACCIONES: COMMIT Y ROLLBACK

EJEMPLO:

```
DROP TABLE IF EXISTS Medico;
CREATE TABLE Medico (
    codigo INT,
    nombre VARCHAR(50),
    apellidos varchar(60),
    especialidad ENUM("cardiología","pediatría","cirugía")
);
INSERT INTO Medico VALUES (1,'Pepe','Rodriguez','cardiología');

SELECT * FROM Medico;

COMMIT;

SET AUTOCOMMIT = 0;

UPDATE Medico SET apellidos = ‘Ape_Modificado’ WHERE codigo = 1;

SELECT * FROM Medico;

ROLLBACK;
```

TRANSICIONES

POLÍTICAS DE BLOQUEO

Cuando una transacción accede a los datos lo hace de forma exclusiva, de modo que una transacción no podrá acceder a los datos

que están siendo utilizados por una transacción hasta que ésta haya terminado.

El bloqueo de los datos se puede realizar a nivel de:

- Base de datos.
- Tabla.
- Fila.
- Columna.

InnoDB realiza por defecto un bloqueo a nivel de fila.